LeetCode HOT 100

PART 11

Binary Search

二分查找

63. 搜索插入位置 (No.35) EASY

给定一个排序数组和一个目标值，在数组中找到目标值，并返回其索引。如果目标值不存在于数组中，返回它将会被按顺序插入的位置。

请必须使用时间复杂度为 `O(log n)` 的算法。

示例 1:

> **输入：** nums = [1,3,5,6], target = 5
> **输出：** 2

示例 2:

> **输入：** nums = [1,3,5,6], target = 2
> **输出：** 1

示例 3:

> **输入：** nums = [1,3,5,6], target = 7
> **输出：** 4

提示：

- $1 <= nums.length <= 10^4$
- $-10^4 <= nums[i] <= 10^4$
- `nums` 为 **无重复元素** 的 **升序** 排列数组
- $-10^4 <= target <= 10^4$

63. 搜索插入位置 (No.35) EASY

思路

这题非常重要，写二分查找插入位置一开始会很喜欢单判left=right的情况(和判断target是否出现不同)或者写的五花八门。这题作为后面部分题目的固定框架，建议固定闭区间/半开半闭区间/开区间三种写法的一种（个人一直都用的闭区间写法因此每轮循环的查找范围是nums[left:right+1]），并按照下面的代码返回left从而保证返回结果一定是从左往右第一个target。

循环过程中保证left左边一定比target小，right右边一定不小于target（但不能保证right及左边没有target，可能有好多相同的target），当缩到一个元素left=right时，①如果击中，返回left成立，②如果target更小说明target就该插入当前位置，返回left成立，③如果target更大说明target应该插入后一个位置，因此left=mid+1返回成立。总之返回left。

```python
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums)-1  # 闭区间写法
        while left <= right:  # 注意等号，否则漏判单元素情况
            mid = (left + right) // 2
            if nums[mid] < target:  # 循环不变量1:nums[left-1] < target
                left = mid + 1
            else:  # 循环不变量2:nums[right+1] >= target
                right = mid - 1
        return left  # 如果hit且target有多个，left必定是第一个target
```

时间复杂度：$\mathcal{O}(\log n)$；**空间复杂度**：$\mathcal{O}(1)$

64. 搜索二维矩阵 (No.74) MEDIUM

给你一个满足下述两条属性的 `m x n` 整数矩阵：

- 每行中的整数从左到右按非严格递增顺序排列。
- 每行的第一个整数大于前一行的最后一个整数。

给你一个整数 `target` ，如果 `target` 在矩阵中，返回 `true` ；否则，返回 `false` 。

提示：

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 100`
- `-10^4 <= matrix[i][j], target <= 10^4`

示例 1：

1	3	5	7
10	11	16	20
23	30	34	60

输入： `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3`
输出： `true`

示例 2：

1	3	5	7
10	11	16	20
23	30	34	60

输入： `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13`
输出： `false`

64. 搜索二维矩阵 (No.74) MEDIUM

Solution 1 – 二分查找

思路

把矩阵展开成一维数组就跟一维的二分查找一样了。这题只需要判断target是否存在，而非通过二分查找确定位置，因此特判击中target的情况，用的经典二分查找模板。

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m*n-1
        while left <= right:
            mid = (left + right) // 2
            row, col = mid // n, mid % n
            if matrix[row][col] == target:
                return True
            elif matrix[row][col] < target:
                left = mid + 1
            else:
                right = mid - 1
        return False
```

时间复杂度：$\mathcal{O}(\log n)$；**空间复杂度**：$\mathcal{O}(1)$

64. 搜索二维矩阵 (No.74) MEDIUM

Solution 2 – 排除法

思路

和[21.搜索二维矩阵II]一样，也可以从矩阵右上角开始用排除法做，target如果更大往下找，target更小往左找，直到下标越界就返回False。

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        i, j = 0, n-1
        while 0 <= i < m and 0 <= j < n:
            if matrix[i][j] == target:
                return True
            elif matrix[i][j] < target:
                i += 1  # target更大往下找
            else:
                j -= 1  # target更小往左找
        return False
```

时间复杂度：$\mathcal{O}(m+n)$；**空间复杂度**：$\mathcal{O}(1)$

65. 在排序数组中查找元素的第一个和最后一个位置 (No.34) MEDIUM

给你一个按照非递减顺序排列的整数数组 `nums`，和一个目标值 `target`。请你找出给定目标值在数组中的开始位置和结束位置。

如果数组中不存在目标值 `target`，返回 `[-1, -1]`。

你必须设计并实现时间复杂度为 `O(log n)` 的算法解决此问题。

示例 1：

> **输入：** nums = [5,7,7,8,8,10], target = 8
> **输出：** [3,4]

示例 2：

> **输入：** nums = [5,7,7,8,8,10], target = 6
> **输出：** [-1,-1]

示例 3：

> **输入：** nums = [], target = 0
> **输出：** [-1,-1]

提示：

- $0 <= nums.length <= 10^5$
- $-10^9 <= nums[i] <= 10^9$
- `nums` 是一个非递减数组
- $-10^9 <= target <= 10^9$

65. 在排序数组中查找元素的第一个和最后一个位置 (No.34) MEDIUM

思路

用[63.搜索插入位置]的模板即可，找到target的第一个下标和target+1的第一个下标(target+1可能不存在，但这个模板返回的是插入位置所以不影响)。如果start下标不合法或者该位置不是target要返回[-1,-1]。

```
class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        def binarySearch(target):
            left, right = 0, n-1
            while left <= right:
                mid = (left + right) // 2
                if nums[mid] < target:
                    left = mid + 1
                else:
                    right = mid - 1
            return left

        n = len(nums)
        start = binarySearch(target)
        end = binarySearch(target+1)
        return [start, end-1] if start < n and nums[start] == target else [-1, -1]
```

时间复杂度：$\mathcal{O}(\log n)$；**空间复杂度**：$\mathcal{O}(1)$

66. 搜索旋转排序数组 (No.33) MEDIUM

整数数组 `nums` 按升序排列，数组中的值 **互不相同** 。

在传递给函数之前，`nums` 在预先未知的某个下标 `k` （`0 <= k < nums.length`）上进行了 **向左旋转**，使数组变为 `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` （下标 **从 0 开始** 计数）。例如， `[0,1,2,4,5,6,7]` 下标 `3` 上向左旋转后可能变为 `[4,5,6,7,0,1,2]` 。

给你 **旋转后** 的数组 `nums` 和一个整数 `target` ，如果 `nums` 中存在这个目标值 `target` ，则返回它的下标，否则返回 `-1` 。

你必须设计一个时间复杂度为 `O(log n)` 的算法解决此问题。

示例 1：

> **输入：** `nums = [4,5,6,7,0,1,2], target = 0`
> **输出：** `4`

示例 2：

> **输入：** `nums = [4,5,6,7,0,1,2], target = 3`
> **输出：** `-1`

示例 3：

> **输入：** `nums = [1], target = 0`
> **输出：** `-1`

提示：

- `1 <= nums.length <= 5000`
- `-10^4 <= nums[i] <= 10^4`
- `nums` 中的每个值都 **独一无二**
- 题目数据保证 `nums` 在预先未知的某个下标上进行了旋转
- `-10^4 <= target <= 10^4`

66. 搜索旋转排序数组 (No.33) MEDIUM

Solution 1 – 一次二分

思路

本题可以使用判断target是否存在的经典模板，hit的情况特判。关键要判断当前mid在前半段还是后半段。如果mid在前半段，target只有在(nums[-1],nums[mid])范围内才在前半段所以right左移，否则left右移；如果mid在后半段，target只有在(nums[mid],nums[-1]]范围内才在后半段所以left右移，否则right左移。

时间复杂度：$\mathcal{O}(\log n)$

空间复杂度：$\mathcal{O}(1)$

```python
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums)-1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            elif nums[mid] > nums[-1]:  # 比末尾元素大，在前半段
                if nums[-1] < target < nums[mid]:  # target在前半段mid的左边
                    right = mid - 1
                else:  # target在前半段mid的右边或者后半段
                    left = mid + 1
            else:  # 不大于末尾元素，在后半段
                if nums[mid] < target <= nums[-1]:  # target在后半段mid的右边
                    left = mid + 1
                else:  # target在后半段mid的左边或前半段
                    right = mid - 1
        return -1
```

66. 搜索旋转排序数组 (No.33) MEDIUM

Solution 2 – 两次二分

思路

这题也可以用两次二分查找，第一次先找到原始的起点下标(旋转前的第一个数下标)，再比较target和末尾元素的大小决定在前半段搜索还是在后半段进行第二次二分查找。

第一次二分查找和一般的二分模式不太一样，由于前半段和后半段都是递增的，因此如果mid元素比末尾元素大说明在前半段，left右移；反之说明当前mid在后半段，right左移。最后仍返回left（其实就是返回不取等的条件分支对应的指针）。

时间复杂度：$\mathcal{O}(\log n)$

空间复杂度：$\mathcal{O}(1)$

```python
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        def binarySearch(left, right, target):
            while left <= right:
                mid = (left + right) // 2
                if nums[mid] < target:
                    left = mid + 1
                else:
                    right = mid - 1
            return left if left < n and nums[left] == target else -1

        def findMin():  # 返回最小元素下标(旋转前的起点)
            left, right = 0, n
            while left <= right:
                mid = (left + right) // 2
                if nums[mid] > nums[-1]:  # 大于末尾元素往右找
                    left = mid + 1
                else:  # 小于等于末尾元素的往左找
                    right = mid - 1
            return left

        n = len(nums)
        start = findMin()
        if target > nums[-1]:  # target在前半段
            return binarySearch(0, start-1, target)
        else:  # target在后半段
            return binarySearch(start, n-1, target)
```

67. 寻找旋转排序数组的最小值 (No.153) MEDIUM

已知一个长度为 `n` 的数组，预先按照升序排列，经由 `1` 到 `n` 次 **旋转** 后，得到输入数组。例如，原数组 `nums = [0,1,2,4,5,6,7]` 在变化后可能得到：

- 若旋转 `4` 次，则可以得到 `[4,5,6,7,0,1,2]`
- 若旋转 `7` 次，则可以得到 `[0,1,2,4,5,6,7]`

注意，数组 `[a[0], a[1], a[2], ..., a[n-1]]` **旋转一次** 的结果为数组 `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]` 。

给你一个元素值 **互不相同** 的数组 `nums` ，它原来是一个升序排列的数组，并按上述情形进行了多次旋转。请你找出并返回数组中的 **最小元素** 。

你必须设计一个时间复杂度为 `O(log n)` 的算法解决此问题。

示例 1：

> **输入：** `nums = [3,4,5,1,2]`
> **输出：** `1`
> **解释：** 原数组为 `[1,2,3,4,5]` ，旋转 `3` 次得到输入数组。

示例 2：

> **输入：** `nums = [4,5,6,7,0,1,2]`
> **输出：** `0`
> **解释：** 原数组为 `[0,1,2,4,5,6,7]` ，旋转 `4` 次得到输入数组。

示例 3：

> **输入：** `nums = [11,13,15,17]`
> **输出：** `11`
> **解释：** 原数组为 `[11,13,15,17]` ，旋转 `4` 次得到输入数组。

提示：

- `n == nums.length`
- `1 <= n <= 5000`
- `-5000 <= nums[i] <= 5000`
- `nums` 中的所有整数 **互不相同**
- `nums` 原来是一个升序排序的数组，并进行了 `1` 至 `n` 次旋转

67. 寻找旋转排序数组的最小值 (No.153) MEDIUM

思路

这就是[66.搜索旋转排序数组]Solution2中找最小值的方法，通过和末尾元素比较判断在前半段还是后半段，前半段left右移，后半段right左移，返回比较时不取等的分支指针left，只不过这题最后要返回的是元素。

时间复杂度：$\mathcal{O}(\log n)$

空间复杂度：$\mathcal{O}(1)$

```python
class Solution:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums)-1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] > nums[-1]:  # 在前半段，右移
                left = mid + 1
            else:  # 在后半段，左移
                right = mid - 1
        return nums[left]
```

68. 寻找两个正序数组的中位数 (No.4) HARD

给定两个大小分别为 `m` 和 `n` 的正序（从小到大）数组 `nums1` 和 `nums2`。请你找出并返回这两个正序数组的 **中位数** 。

算法的时间复杂度应该为 `O(log (m+n))` 。

示例 1：

> **输入：** `nums1 = [1,3], nums2 = [2]`
> **输出：** `2.00000`
> **解释：** `合并数组 = [1,2,3] ，中位数 2`

示例 2：

> **输入：** `nums1 = [1,2], nums2 = [3,4]`
> **输出：** `2.50000`
> **解释：** `合并数组 = [1,2,3,4] ，中位数 (2 + 3) / 2 = 2.5`

提示：

- `nums1.length == m`
- `nums2.length == n`
- `0 <= m <= 1000`
- `0 <= n <= 1000`
- `1 <= m + n <= 2000`
- -10^6 `<= nums1[i], nums2[i] <=` 10^6

68. 寻找两个正序数组的中位数 (No.4) HARD

思路

个人认为这题是HOT100里面最难的一题。

思路其实就是用二分查找去找两个数组里面第k小的数，k是两个数组总长+1的一半，+1是为了统一奇偶的情况。

对于nums1和nums2，它们都应有各自的“较小组”，整个较小组合起来k个数，因此如果nums1中有mid1+1个数，nums2中就有k-mid1-1个数。

在二分查找的时候，用mid1表示nums1较小组末尾下标，mid2表示nums2较大组起始下标，必须保证较小组的最大值大于较大组的最小值。和[63.插入搜索位置]一样，left返回nums[mid2]插入的地方，也就是mid1较大组的起始下标。因此nums1较小组末尾下标是left-1，从而nums2是k-left-1。

```python
class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        n1, n2 = len(nums1), len(nums2)
        if n1 > n2:  # 保证二分的一定是较短的数组
            return self.findMedianSortedArrays(nums2, nums1)
        k = (n1 + n2 + 1) // 2  # 中位数下标，奇偶数统一处理，否则奇数长度k还要+1
        left, right = 0, n1 - 1
        while left <= right:
            mid1 = (left + right) // 2  # mid1是nums1中较小组的末尾下标
            mid2 = k-mid1-1  # mid2是nums2中较大组起始下标，也是较小组数量k-(mid1+1)
            # nums1[mid1] < nums2[mid2]，符合较小组定义，mid1还可以增大右移
            if nums1[mid1] < nums2[mid2]:
                left = mid1 + 1
            else:  # nums[mid1] >= nums[mid2] 不符合较小组定义，mid1收缩左移
                right = mid1 - 1
        # 退出循环后left是nums2[mid2]在nums1中应插入的位置，因此left-1就是较小组末尾
        mid1, mid2 = left-1, k-left-1  # nums2较小组末尾下标应为k-(mid1+1)-1
        # 较小组最大值，注意处理较小组为空的情况
        c1 = max(nums1[mid1] if mid1>=0 else -inf,
                 nums2[mid2] if mid2>=0 else -inf)
        # 较大组最小值，注意处理较大组为空的情况
        c2 = min(nums1[mid1+1] if mid1+1<n1 else inf,
                 nums2[mid2+1] if mid2+1<n2 else inf)
        return c1 if (n1+n2)%2 else (c1+c2)/2
```

我这里的仍然是闭区间的写法

时间复杂度：$\mathcal{O}(\log min(m+n))$；**空间复杂度**：$\mathcal{O}(1)$